

DC-Leap: Training-Free Acceleration of dLLMs via Draft-Guided Contiguous Leaping Decoding

Yanhua Jiao^{1,2} Tianyi Wu¹ Xiaoxi Sun¹ Yulin Li¹
Huiling Zhen³ Libo Qin¹ Baotian Hu^{1,2} Zhuotao Tian^{† 1,2} Min Zhang^{1,2}

Abstract

While parallel decoding is central to the efficiency of Diffusion Large Language Models (dLLMs), current strategies are often hindered by overly conservative confidence thresholds. These thresholds, necessitated by the Joint Probability Dependence Error (JPDE), result in redundant denoising iterations and suboptimal inference speeds. To overcome this, we propose DC-Leap, a training-free framework that enables reliable acceleration of dLLMs in the moderate-confidence regime. DC-Leap introduces a Dynamic Contiguous Verification strategy that integrates strictly-ordered causal constraints into the parallel decoding process. By progressively validating token dependencies, this mechanism effectively neutralizes the JPDE, enabling reliable acceleration with comparable performance. Furthermore, DC-Leap incorporates the draft-guided decoding mechanism, where the draft helps extend the context by leaping forward across multiple tokens, providing look-ahead context and retaining the structural benefits of bidirectional attention during inference. Extensive experiments on standard benchmarks demonstrate that DC-Leap achieves substantial speedups, up to **53.19×** on MBPP for long-sequence generation, and up to **105.02×** when combined with KV-Cache with comparable generation quality. Code is available at <https://github.com/ffh-wyls/DC-Leap>.

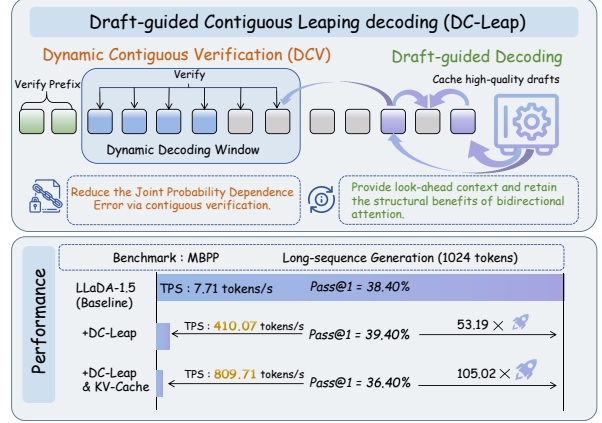


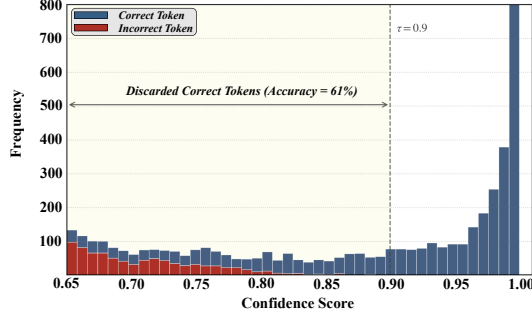
Figure 1. Overview and Performance of DC-Leap. Top: Schematic of the proposed training-free framework, featuring Dynamic Contiguous Verification to neutralize JPDE through causal constraints, and a Draft Mechanism to expedite convergence via distal context. Bottom: Evaluation on MBPP for long-sequence generation. DC-Leap achieves up to **53.19×** significant speedup while maintaining near-lossless performance.

1. Introduction

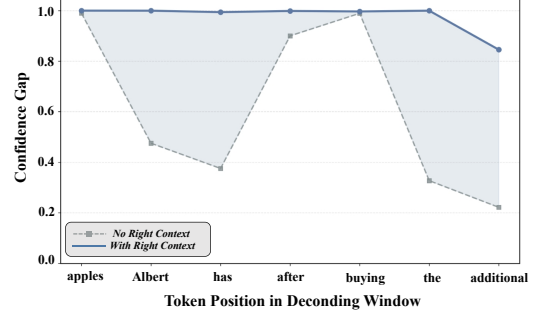
Diffusion Large Language Models (dLLMs) such as LLaDA (Nie et al., 2025; Zhu et al., 2025) and Dream (Ye et al., 2025), have emerged as a compelling generative paradigm, challenging the dominance of Autoregressive (AR) models by offering non-autoregressive capabilities and achieving generation quality comparable to models of similar scale. Unlike the strict sequential dependency inherent in AR architectures, dLLMs inherently support parallel decoding, presenting a viable path toward overcoming the latency bottlenecks associated with sequential token generation. However, dLLMs may often resort to redundant iterative refinement, resulting in inference throughput that trails behind highly optimized AR models, thereby limiting the utility of dLLMs in latency-sensitive applications.

Motivation. To mitigate dLLM’s computational burden, recent research has focused on parallel decoding strategies, such as Fast-dLLM (Wu et al., 2026) and LocalLeap (Kong et al., 2025). These methods accelerate decoding by accept-

¹Harbin Institute of Technology, Shenzhen ²Shenzhen Loop Area Institute ³Huawei Noah’s Ark Lab. Correspondence to: Zhuotao Tian [†] <tianzhuotao@hit.edu.cn>.



(a) The frequency of correct versus incorrect predictions across different confidence intervals from LLaDA-1.5. Tokens in the loose confidence range (0.65–0.9) account for a significant portion of correct predictions yet are discarded by conservative thresholds.



(b) Confidence gap ($P_{\text{top1}} - P_{\text{top2}}$) for intermediate tokens. The presence of right-context (blue line) significantly widens the margin relative to the baseline (grey line). This trend demonstrates that future information sharpens the predictive distribution, thereby reducing ambiguity during the generation of intermediate tokens.

Figure 2. **Motivation of DC-Leap.** (a) Strict confidence thresholds in existing parallel decoding methods lead to the waste of potentially correct tokens. (b) Leveraging right-side context reduces the confidence gap, facilitating faster convergence.

ing multiple tokens per step based on the confidence thresholds. Typically, these methods use strict, high-confidence thresholds, which are usually above 0.9. However, as depicted in Figure 2 (a), a substantial fraction of correct tokens (61% within the 0.65–0.9 confidence interval) are often discarded due to falling below the predefined confidence threshold. This premature filtering can create an efficiency bottleneck by restricting the model’s capacity to leverage rapid token convergence in lower-confidence regions, thus limiting the potential acceleration gains.

In fact, such stringent thresholding primarily arises from the need to manage errors introduced by parallel acceptance strategies: their effectiveness depends on the assumption that *token predictions are conditionally independent given the current context*. However, recent findings by Ben-Hamu et al. (2026) highlight that parallel decoding inherently induces the *Joint Probability Dependence Error (JPDE)*, caused by simultaneously predicting multiple tokens while neglecting their mutual dependencies, resulting in incoherent outputs. Thus, a key research question emerges: *Can we mitigate the dependence-induced errors, i.e., JPDE, through alternative methods, beyond stringent confidence thresholding, allowing for safely reduced thresholds and consequently greater acceleration?*

Limitations of Sequential Decoding. A natural approach to mitigating JPDE is to adopt sequential decoding, where tokens are committed one-by-one, conditioning each prediction explicitly on previously accepted tokens. In principle, this strategy resolves dependence errors by avoiding simultaneous token predictions within the same iteration; instead, joint predictions across tokens are factorized through the chain rule. However, sequential decoding faces two fundamental limitations in the context of dLLMs.

First, it suffers from *prohibitive inefficiency*: sequential commitment reduces parallelism, causing the number of forward passes, and consequently latency, to scale linearly with sequence length and severely undermining the acceleration effects.

Second, sequential decoding yields an *unfavorable inference regime for bidirectional dLLMs*: the structured, typically left-to-right conditioning pattern inherent to sequential prediction deviates from the distribution encountered during the bidirectional denoising training of dLLMs, as evidenced by the minor confidence gap for most tokens in the No Right Context (fully masked) setting in Figure 2 (b).

Such a mismatch limits the exploitation of bidirectional context at inference, as well as the performance gains brought by bidirectional attention mechanisms.

Our Solution. To mitigate JPDE while preserving the efficiency of parallel decoding and the advantages of bidirectional context in dLLMs, we introduce **DC-Leap**, a training-free acceleration strategy via **Draft-guided Contiguous Leaping** decoding. DC-Leap draws inspiration from sequential commitment but circumvents its two major limitations. It consists of two primary mechanisms: 1) *Dynamic Contiguous Verification (DCV)* and 2) *Draft-guided Decoding*.

Specifically, DCV introduces the *dynamic decoding window* that enforces a local, sequential left-to-right verification of tokens. Within each window, tokens are accepted contiguously until a prediction falls below the confidence threshold, thereby replacing the global conditional independence assumption inherent to parallel decoding with a local, window-level factorization, enabling multi-token acceptance while still retaining the efficiency.

On the other hand, to address the distributional mismatch

and context limitations caused by the sequential commitment, DC-Leap maintains non-committed *drafts*, i.e., high-confidence tokens predicted outside the decoding window. These drafts help extend the context by *leaping forward* across multiple tokens, providing look-ahead context and retaining the structural benefits of bidirectional attention during inference.

Extensive experiments conducted on three dLLMs across five benchmarks, including math reasoning, instruction following, and code generation, demonstrate the effectiveness and the generalization capabilities of DC-Leap. Our primary contributions can be summarized as follows:

- We identify a key trade-off in dLLM acceleration: controlling Joint Probability Dependence Error (JPDE) necessitates strict thresholds in parallel decoding, whereas sequential decoding reduces JPDE at the expense of efficiency and bidirectional inference.
- We propose **DC-Leap**, an effective decoding framework that integrates dynamic continuous verification with a look-ahead draft mechanism, safely lowering the acceptance threshold for parallel decoding while exploiting bidirectional attention during inference.
- Extensive experiments on LLaDA-1.5, LLaDA-8B-Instruct, and Dream-v0-7B-Instruct models across five benchmarks validate the effectiveness of DC-Leap, demonstrating substantial inference speedups without compromising the generation quality of the baselines.

2. Background and Motivation

In this section, we introduce the preliminaries of dLLMs in Sec. 2.1 and present our empirical findings in Sec. 2.2, which motivate the designs of our proposed framework.

2.1. Preliminaries

Parallel Decoding in dLLMs. Consider a generation task aiming to produce a target sequence $\mathbf{x}_{\text{gen}}$ of length M , conditioned on a prompt $\mathbf{x}_{\text{prompt}}$ of length N . The complete sequence is denoted by $\mathbf{x} \in \mathcal{V}^L$, with $L = N + M$ and vocabulary $\mathcal{V}$. Let $[\text{MASK}] \in \mathcal{V}$ represent the mask token.

Unlike autoregressive methods, dLLMs (Nie et al., 2025) employ a non-autoregressive iterative denoising process over a discrete time horizon $t \in \{T, T-1, \dots, 0\}$, which typically starts at step T with a fully masked response appended to the prompt:

$$\mathbf{x}^T = (\mathbf{x}_{\text{prompt}}, \underbrace{[\text{MASK}], \dots, [\text{MASK}]}_{M \text{ times}}). \quad (1)$$

At each denoising iteration t , the model p_θ simultaneously estimates the categorical distribution over the vocabulary for

all masked positions. Specifically, for any position i where $\mathbf{x}_i^t = [\text{MASK}]$, the model computes token probabilities:

$$P(\mathbf{x}_i = v \mid \mathbf{x}^t) = [\text{Softmax}(p_\theta(\mathbf{x}^t)_i)]_v, \quad \forall v \in \mathcal{V}. \quad (2)$$

Parallel decoding methods for dLLMs (Wu et al., 2026; Ben-Hamu et al., 2026) typically employ a confidence-based thresholding strategy to transition from $\mathbf{x}^t$ to $\mathbf{x}^{t-1}$. Specifically, for each masked token position i , a binary acceptance decision M_i is made independently as follows:

$$M_i = \mathbb{I} \left(\max_{v \in \mathcal{V}} P(\mathbf{x}_i = v \mid \mathbf{x}^t) > \tau \right), \quad (3)$$

where $\tau \in [0, 1]$ represents the pre-defined confidence threshold. Tokens satisfying $M_i = 1$ are committed simultaneously in a single iteration.

Joint Probability Dependence Error in Parallel Decoding. Parallel decoding strategies accelerate inference by simultaneously predicting multiple masked tokens indexed by the set U . These methods approximate the true joint distribution $P(\mathbf{x}_U \mid \mathcal{C})$ with the product of marginals $Q(\mathbf{x}_U \mid \mathcal{C}) = \prod_{i \in U} p_\theta(\mathbf{x}_i \mid \mathcal{C})$, where $\mathcal{C}$ is the context that has already been decoded.

This conditional independence assumption introduces the Joint Probability Dependence Error ($\mathcal{E}$). As derived in (Ben-Hamu et al., 2026), this error can be quantified by the KL divergence, which is strictly bounded above by the marginal entropies of the target tokens:

$$\mathcal{E} = D_{\text{KL}}(P \parallel Q) \leq \sum_{i \in U} H(\mathbf{x}_i \mid \mathcal{C}) - \max_{j \in U} H(\mathbf{x}_j \mid \mathcal{C}). \quad (4)$$

For non-contiguous decoding, the sparse context $\mathcal{C}_0$ composed of scattered verified tokens yields high accumulated entropies and loose error bounds. Consequently, these methods compulsorily require conservative confidence thresholds τ in Eq. (3) to filter unreliable predictions, severely diminishing their acceleration potential.

2.2. Key Observations

As discussed in the Introduction, parallel decoding is fundamentally constrained by JPDE, thus relying on strict confidence thresholds to minimize errors. However, this conservatism leads to excessive rejection of correct tokens in early iterations (Figure 2 (a)). This raises the core challenge: *how can we safely lower the threshold to recover these tokens without compromising accuracy?*

The problem can be equivalently reformulated as mitigating the JPDE discussed in Sec. 2.1 without relying on stringent thresholds. We draw inspiration from sequential decoding, which is theoretically free from JPDE as it strictly follows

Table 1. Accuracy (%) on GSM8K in the LLaDA-1.5 under different confidence thresholds τ for two decoding strategies.

Threshold τ	0.60	0.65	0.70	0.75
Contiguous	77.63	78.85	79.53	80.59
Non-Contiguous	75.13	77.48	79.08	80.44

the probability chain rule:

$$P(\mathbf{x}_{1:T} | \mathcal{C}) = \prod_{t=1}^T P(x_t | \mathbf{x}_{<t}, \mathcal{C}). \quad (5)$$

where $\mathbf{x}_{1:T}$ represents the target generation sequence of total length T , and $\mathcal{C}$ denotes the given context. The x_t refers to the specific token generated at the current time step t , which is explicitly conditioned on the fully verified prefix $\mathbf{x}_{<t} = \{x_1, \dots, x_{t-1}\}$ accumulated from previous steps. This ensures that the independence assumption is never violated, thus guaranteeing the generation quality.

Windowed Contiguous Decoding. Although sequential decoding avoids JPDE by construction, its token-by-token commitment is prohibitively inefficient. This motivates a natural question: *Can we retain left-to-right sequentiality while decoding multiple tokens per iteration?*

A promising compromise involves relaxing strict sequentiality to the span level. Consider decoding a candidate span of tokens in parallel, while adhering to a left-to-right commitment rule: only the longest verified contiguous prefix is accepted, meaning that a token is retained only if all preceding tokens in the span are verified. Specifically, as shown in Table 1, contiguous decoding boosts accuracy compared to non-contiguous alternatives under the same confidence thresholds. This demonstrates that strict sequentiality can coexist with multi-token commitment, effectively reconciling orderly verification with computational efficiency.

Future Context Benefits. Sequential verification enforces a left-to-right commitment pattern, which leaves most future positions masked during decoding. As a result, dLLMs cannot fully exploit their bidirectional attention, since informative right context is largely unavailable at inference time. This motivates a natural question: *Can supplementing reliable right-side future context improve the model’s confidence and stability under verification?*

To validate this, we conduct a simple pilot study that provides additional right-context information during decoding and quantifies its effect on prediction uncertainty. Specifically, we measure the confidence gap $P_{\text{top1}} - P_{\text{top2}}$ under varying right-context conditions, where P_{top1} and P_{top2} denote the probabilities of the most likely and second-most likely tokens, respectively. As shown in Figure 2 (b), supplying high-quality right context consistently sharpens the

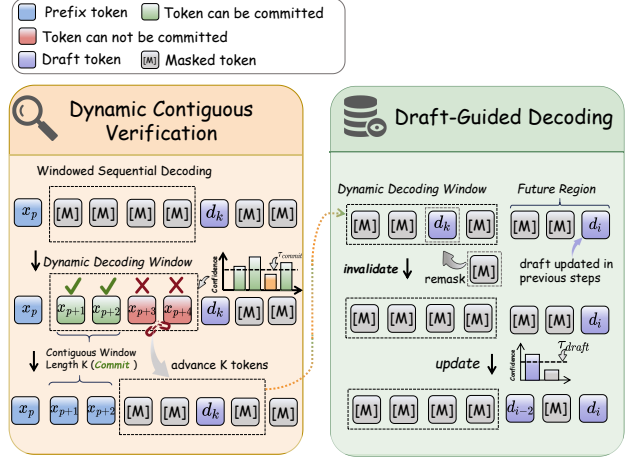


Figure 3. **Dynamic Contiguous Verification (DCV) and Draft Mechanisms.** Left: DCV determines window length K (tokens exceeding τ_{commit}) and advances by committing K contiguous tokens. Right: Draft mechanism remarks invalid tokens within the window and selects future tokens on the right side of the current window exceeding τ_{draft} as placeholders for subsequent decoding.

confidence gap compared to decoding without right context, indicating more decisive token predictions.

This suggests that future tokens can serve as semantic anchors that reduce uncertainty and stabilize verification. Moreover, since higher confidence is typically correlated with a higher likelihood of correctness (Guo et al., 2017), confidence provides a practical proxy for selecting useful future cues. These observations motivate our draft mechanism, which retains high-confidence future predictions as look-ahead placeholders to guide decoding.

3. Methodology

3.1. Overview

Inspired by insights discussed in Sec. 2.2, we propose DC-Leap to safely lower the confidence threshold used in parallel decoding, thus enabling significant inference acceleration for dLLMs. As illustrated in Figure 3, DC-Leap addresses the independence-induced errors common in conventional parallel decoding by incorporating two key components.

First, we introduce *Dynamic Contiguous Verification* (Sec. 3.2). This strategy adopts a dynamic decoding window that advances sequentially, while tokens within each window are decoded in parallel. By enforcing sequential dependencies at the window level rather than globally, DC-Leap relaxes the restrictive global conditional independence assumption inherent in parallel decoding. As a result, a more lenient confidence threshold can be safely used, allowing more tokens to be accepted at each iteration.

Secondly, we address the issue raised in Sec. 2.2 that contiguous decoding may lead bidirectional dLLMs into a sub-

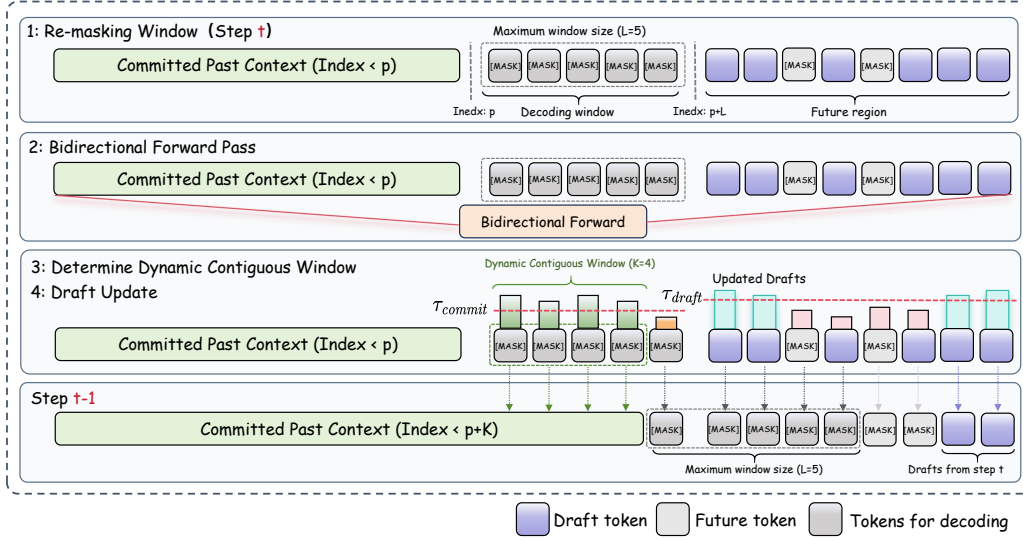


Figure 4. **Illustration of the draft-guided decoding process.** The proposed pipeline reconciles window-level sequentiality with global contextual awareness. By combining dynamic contiguous verification with an in-place draft update mechanism, DC-Leap utilizes future context on the right side to guide current denoising, effectively alleviating context limitations induced by strictly sequential decoding.

optimal inference regime by introducing the *draft* mechanism (Sec. 3.3). Drafts are high-confidence, decoded yet uncommitted tokens outside the current decoding window. These drafts provide valuable look-ahead future context, enabling dLLMs to effectively leverage bidirectional attention capabilities for decoding.

The comprehensive algorithmic procedure of DC-Leap is systematically detailed in Algorithm A.

3.2. Dynamic Contiguous Verification

To mitigate Joint Probability Dependence Error (JPDE) without the aggressive confidence filtering used in prior works (Wu et al., 2026; Kong et al., 2025), we propose *Dynamic Contiguous Verification (DCV)*. DCV replaces the global conditional independence assumption inherent to parallel decoding with a local, window-level factorization. By sequentially advancing a contiguous window while decoding all tokens within it in parallel, DCV retains local sequential constraints to mitigate JPDE without sacrificing efficiency. This structural refinement allows the model to adopt a significantly more lenient confidence threshold, effectively capturing acceleration opportunities in lower-confidence regions that were previously suppressed.

Dynamic Decoding Window. We define the dynamic decoding window (DDW) as the set of tokens that will be decoded and committed at the current iteration. Unlike fixed-length windows, the span of DDW is dynamically determined from the model’s current predictions, subject to a maximum window size L .

Formally, let p denote the leftmost uncommitted position in the sequence. At the current iteration, the decoding window is a contiguous segment $W = [p, p + K)$, where the window length K satisfies $0 \leq K \leq L$.

To dynamically determine the window length K based on the token predictions, we introduce a commitment threshold τ_{commit} . A token at position i is eligible for commitment if its confidence $c_i \geq \tau_{\text{commit}}$. However, unlike standard parallel decoding schemes that commit all tokens exceeding the threshold in a single step, we enforce contiguous commitment: only the longest prefix starting at position p whose tokens all satisfy the confidence constraint is committed.

Consequently, within the maximum window size L , the window length K is determined by truncating at the first position where the confidence falls below τ_{commit} . For implementation convenience, we compute K using an indicator-based prefix-product formulation:

$$K = \sum_{i=0}^{L-1} \prod_{j=0}^i \mathbb{I}(c_j > \tau_{\text{commit}}), \quad (6)$$

which counts the length of the longest contiguous prefix whose token confidences exceed the commit threshold.

This sequential advancement of the contiguous and monotonic window ensures that tokens are verified in their correct order. By preventing out-of-order commitments, the mechanism allows DCV to safely adopt a more lenient confidence threshold, thereby maximizing the number of tokens accepted in each parallel decoding iteration.

Table 2. **Performance on Dream-v0-7B-Instruct (Ye et al., 2025).** Each cell reports accuracy and throughput (TPS), alongside the speedup factor relative to the baseline. Bold indicates the highest performance.

	MATH (4-shot)	MBPP (3-shot)	GSM8K (5-shot)	IFEval (0-shot)	HumanEval (0-shot)	Avg.
Dream-v0-7B-Instruct	45.44	53.40	75.36	48.92	54.88	55.60
Throughput (TPS, $\uparrow$)	16.87 (1.0 $\times$)	17.37 (1.0 $\times$)	13.92 (1.0 $\times$)	27.25 (1.0 $\times$)	20.17 (1.0 $\times$)	1.0 $\times$
+ Fast-dLLM	44.74	53.40	74.75	47.24	56.71	55.37
Throughput (TPS, $\uparrow$)	46.11 (2.73 $\times$)	46.41 (2.67 $\times$)	44.99 (3.23 $\times$)	38.30 (1.41 $\times$)	37.32 (1.85 $\times$)	2.38 $\times$
+ LocalLeap	44.66	57.00	73.92	50.00	56.71	56.46
Throughput (TPS, $\uparrow$)	51.44 (3.05 $\times$)	81.75 (4.71 $\times$)	57.68 (4.14 $\times$)	77.37 (2.84 $\times$)	83.26 (4.13 $\times$)	3.77 $\times$
+ L2P	44.98	51.60	73.92	45.76	58.54	54.96
Throughput (TPS, $\uparrow$)	20.37 (1.21 $\times$)	30.19 (1.74 $\times$)	23.65 (1.70 $\times$)	48.62 (1.78 $\times$)	40.90 (2.03 $\times$)	1.69 $\times$
+ DC-Leap (Ours)	44.18	57.60	73.92	51.92	59.76	57.48
Throughput (TPS, $\uparrow$)	53.70 (3.18$\times$)	88.59 (5.10$\times$)	75.87 (5.45$\times$)	110.64 (4.06$\times$)	116.21 (5.76$\times$)	4.71$\times$

3.3. Draft-guided Decoding

While DCV effectively mitigates JPDE through window-level sequentiality, it inherently restricts the model’s access to right-side context during inference. Such context truncation diminishes the ability of bidirectional dLLMs to exploit future token information, potentially leading to suboptimal inference performance. To overcome this issue, we introduce *Draft-guided Decoding*, a method that maintains uncommitted draft placeholders to provide look-ahead context, thus preserving the structural advantages of bidirectional attention during inference.

Specifically, we generate *drafts* by predicting tokens at select positions beyond the current decoding window, employing them as auxiliary right-side context. These drafts are maintained as *look-ahead placeholders* and are not finalized as outputs, thus preserving window-level sequentiality. In essence, drafts serve as right-side contextual anchors for decoding the current window, aligning inference-time conditions more closely with the bidirectional denoising setting encountered during training, and thereby enhancing the effective utilization of bidirectional attention.

Draft Update. We now formalize the draft update procedure. During the forward pass, predictions in the future region are monitored in parallel. We introduce a draft threshold τ_{draft} and designate positions whose confidence satisfies $c > \tau_{\text{draft}}$ as draft candidates. After committing the tokens within the decoding window, we update the sequence by overwriting the corresponding draft placeholders in place.

Formally, let $\hat{x}_i$ denote the top-1 predicted token at position i in the future region, *i.e.*, $i \geq p + L$, where p is the index of the last verified token and L is the decoding window length. Let c_i denote the confidence score of token $\hat{x}_i$. We maintain a draft placeholder d_i for each future position i . Initially set as a [MASK] token, d_i is iteratively updated as follows:

$$d_i = \begin{cases} \hat{x}_i & \text{if } c_i > \tau_{\text{draft}} \\ d_i & \text{otherwise} \end{cases}, \forall i \geq p + L, \quad (7)$$

Draft Invalidation. However, since drafts are non-committed placeholders, they must be invalidated once they fall within the range of tokens eligible for inclusion in the decoding window. To prevent trivial copying and to preserve window-level sequential decoding, we discard such drafts by re-masking. For simplicity, we re-mask all tokens within this maximum length range, thereby removing any drafts that are potentially incorporated into the decoding window:

$$x_{p+i} \leftarrow [\text{MASK}], \quad \forall i \in [0, L - 1]. \quad (8)$$

Here, p denotes the starting position of the current decoding window, L denotes the maximum window size, and x_{p+i} is the token at position $p + i$ among those eligible for inclusion in the decoding window.

Inference with Drafts. At each decoding iteration, the proposed draft-guided decoding pipeline consists of four steps as shown in Figure 4: (i) re-masking the decoding window to invalidate any drafts it contains as Eq. (8); (ii) performing a single forward pass to obtain predictions for both the window and the future region; (iii) computing the dynamic contiguous window length K as Eq. (6); and (iv) committing the verified window tokens while updating drafts in place as Eq. (7). Overall, the draft-guided decoding pipeline provides look-ahead future context to the decoding region, while still preserving the window-level sequentiality of DCV, thereby alleviating the distributional mismatch and context limitations induced by the sequential decoding.

4. Experiments

4.1. Experiments Setup

Models and Baselines. We select LLaDA-8B-Instruct (Nie et al., 2025), LLaDA-1.5 (Zhu et al., 2025), and Dream-v0-Instruct-7B (Ye et al., 2025) as our base models. For each, the generation follows a semi-autoregressive remasking strategy (Nie et al., 2025) configured with a block size of 32. We compare our DC-Leap against standard diffusion decoding and three state-of-the-art parallel decoding strategies: (1) Fast-dLLM (Wu et al., 2026), a confidence-thresholding

Table 3. Performance on LLaDA-8B-Instruct (Nie et al., 2025). Each cell reports accuracy and throughput (TPS), alongside the speedup factor relative to the baseline. Bold indicates the highest performance.

	MATH (4-shot)	MBPP (3-shot)	GSM8K (5-shot)	IFEval (0-shot)	HumanEval (0-shot)	Avg.
LLaDA-8B-Instruct	38.74	30.00	77.71	70.74	35.37	50.51
Throughput (TPS, $\uparrow$)	3.40 (1.0 $\times$)	12.96 (1.0 $\times$)	10.47 (1.0 $\times$)	21.07 (1.0 $\times$)	19.16 (1.0 $\times$)	1.0 $\times$
+ Fast-dLLM	38.30	30.00	77.94	70.62	35.37	50.45
Throughput (TPS, $\uparrow$)	33.08 (9.73 $\times$)	44.93 (3.47 $\times$)	31.90 (3.05 $\times$)	49.14 (2.33 $\times$)	57.65 (3.01 $\times$)	4.32 $\times$
+ LocalLeap	37.94	30.00	77.33	70.14	34.76	50.03
Throughput (TPS, $\uparrow$)	41.97 (12.34 $\times$)	55.98 (4.32 $\times$)	41.86 (4.00 $\times$)	54.88 (2.60 $\times$)	76.33 (3.98 $\times$)	5.45 $\times$
+ L2P	37.96	30.60	77.41	70.26	34.76	50.20
Throughput (TPS, $\uparrow$)	44.35 (13.04 $\times$)	57.77 (4.46 $\times$)	43.19 (4.13 $\times$)	59.13 (2.81 $\times$)	80.57 (4.21 $\times$)	5.73 $\times$
+ DC-Leap (Ours)	37.28	29.60	76.65	71.34	35.37	50.05
Throughput (TPS, $\uparrow$)	49.82 (14.65$\times$)	59.30 (4.58$\times$)	50.22 (4.80$\times$)	72.48 (3.44$\times$)	90.72 (4.73$\times$)	6.44$\times$

Table 4. Performance on LLaDA-1.5 (Zhu et al., 2025). Each cell reports accuracy and throughput (TPS), alongside the speedup factor relative to the baseline. Bold indicates the highest performance.

	MATH (4-shot)	MBPP (3-shot)	GSM8K (5-shot)	IFEval (0-shot)	HumanEval (0-shot)	Avg.
LLaDA-1.5	38.96	37.40	80.74	66.67	40.24	52.80
Throughput (TPS, $\uparrow$)	3.46 (1.0 $\times$)	13.14 (1.0 $\times$)	10.58 (1.0 $\times$)	21.65 (1.0 $\times$)	19.38 (1.0 $\times$)	1.0 $\times$
+ Fast-dLLM	38.94	37.60	81.05	66.97	39.63	52.84
Throughput (TPS, $\uparrow$)	33.80 (9.77 $\times$)	73.83 (5.62 $\times$)	32.92 (3.11 $\times$)	48.39 (2.24 $\times$)	55.34 (2.86 $\times$)	4.72 $\times$
+ LocalLeap	38.22	39.20	81.58	65.11	40.24	52.87
Throughput (TPS, $\uparrow$)	42.28 (12.22 $\times$)	90.06 (6.85 $\times$)	43.51 (4.11 $\times$)	68.55 (3.17 $\times$)	71.28 (3.68 $\times$)	6.01 $\times$
+ L2P	38.66	37.40	80.89	66.31	39.02	52.46
Throughput (TPS, $\uparrow$)	45.30 (13.09 $\times$)	96.43 (7.34 $\times$)	45.68 (4.32 $\times$)	56.83 (2.62 $\times$)	76.00 (3.92 $\times$)	6.26 $\times$
+ DC-Leap (Ours)	38.66	42.60	80.21	71.82	38.41	54.34
Throughput (TPS, $\uparrow$)	51.72 (14.95$\times$)	171.35 (13.04$\times$)	52.07 (4.92$\times$)	69.67 (3.22$\times$)	102.74 (5.30$\times$)	8.29$\times$

parallel decoding method; (2) LocalLeap (Kong et al., 2025), which leverages high-confidence anchors to guide local parallelization; and (3) L2P (Bao et al., 2026), a learnable policy network designed for adaptive parallel decoding.

Benchmarks and Metrics. To comprehensively assess the performance and efficiency of our method, we conduct evaluations across three primary domains: Mathematical Reasoning (GSM8K (Cobbe et al., 2021), MATH (Hendrycks et al., 2021)), Program Synthesis (HumanEval (Chen et al., 2021), MBPP (Austin et al., 2021)), and Instruction Following (IFEval (Zhou et al., 2023)). We set the generation length to 512 for HumanEval and IFEval, while a length of 256 is applied to all other benchmarks. Efficiency is quantified via Tokens Per Second (TPS) and relative speedup over the baseline.

Implementation Details. All experiments are performed on NVIDIA RTX PRO 6000 Blackwell. To ensure reproducibility and fair comparison, we utilize greedy decoding without any random sampling strategies. Unless otherwise specified, we set the maximum decoding window size to half of the total generation length, which determines the span of tokens being verified. Throughout our experiments, we maintain constant threshold values of $\tau_{\text{commit}} = 0.70$ and $\tau_{\text{draft}} = 0.98$ across all models and benchmarks, demonstrating the robustness and generalization of DC-Leap in diverse evaluation scenarios.

4.2. Main Results

The results are presented in Table 2, 3, and 4. Overall, DC-Leap achieves substantial inference speedup with negligible performance degradation and, in several cases, slight improvement. In terms of decoding speed, DC-Leap consistently outperforms all baselines across diverse models and benchmarks. On Dream-v0-7B-Instruct (Table 2), DC-Leap achieves an average speedup of 4.71 $\times$, significantly surpassing the previous state-of-the-art LocalLeap (3.77 $\times$) and Fast-dLLM (2.38 $\times$). The performance gain is even more noticeable on the LLaDA-1.5 (Table 4). By unlocking the potential of lower-confidence tokens via contiguous verification, DC-Leap attains an average speedup of 8.29 $\times$. Notably, on code generation tasks such as MBPP, DC-Leap achieves a throughput improvement of 13.04 $\times$, demonstrating its exceptional efficiency.

Scalability and Extension. We further evaluate DC-Leap on long sequence generation (1024 tokens) and its compatibility with KV cache on LLaDA-1.5, as shown in Table 5. The results highlight superior scalability, where acceleration gains amplify with sequence length. Specifically, on GSM8K and MBPP, DC-Leap achieves 24.64 $\times$ and 53.18 $\times$ speedup, respectively. Furthermore, DC-Leap proves orthogonal to the KV Cache optimizations. Integrating DC-Leap with dLLM-Cache (Liu et al., 2025) yields a compound acceleration effect, pushing throughput to a 105.02

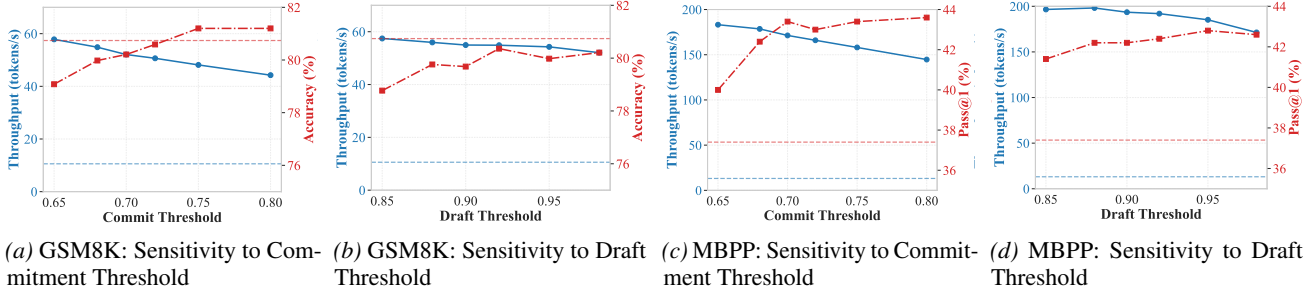


Figure 5. Sensitivity analysis on τ_c and τ_d for LLaDA-1.5 on GSM8K and MBPP benchmarks. We perform univariate analysis for each threshold while fixing the other at the default setting. Blue solid lines and red dash-dotted lines represent throughput (TPS) and task performance (Accuracy or Pass@1), respectively, with horizontal dashed lines marking corresponding baseline levels.

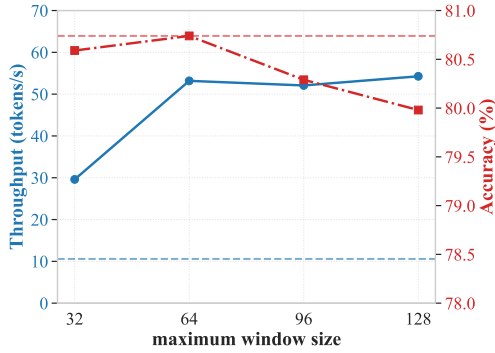


Figure 6. Sensitivity analysis on the maximum window length L for LLaDA-1.5 on GSM8K benchmark. Throughput (TPS, blue solid) and accuracy (red dash-dotted) are plotted against L , with horizontal dashed lines marking their respective baselines.

× (809.71 TPS) on MBPP while maintaining competitive generation quality. This establishes DC-Leap as a robust, plug-and-play module for dLLMs acceleration.

4.3. Ablations and Analysis

To understand hyperparameter sensitivity in DC-Leap, we conduct extensive ablations on LLaDA-1.5 using GSM8K and MBPP, analyzing the impact of dual thresholds (commitment and draft) and maximum window size.

Impact of commitment threshold τ_c . To explore the impact of dynamic contiguous verification, we fix the draft threshold τ_d at 0.98 and vary τ_c . As shown in Figure 5 (a, c), lowering τ_c unlocks significant speedups by allowing more tokens to be accepted within the decoding window. Specifically, on GSM8K, decreasing τ_c from 0.80 to 0.65 boosts throughput from 44.24 to 57.84 TPS without compromising accuracy, demonstrating that DC-Leap is robust to τ_c . A consistent trend is observed in the MBPP results. To strike an optimal balance between accuracy and throughput, we set $\tau_c = 0.70$ as our default configuration.

Table 5. Performance of DC-Leap on 1024-token generation, with and without dLLM-Cache. Best performances are in bold.

	Method	Throughput (TPS, ↑)	Performance (Score %)
GSM8K	LLaDA-1.5	6.63 (1.0×)	82.79
	+ Cache	17.23 (2.60×)	83.09
	+ DC-Leap	163.38 (24.64×)	82.34
	+ DC-Leap + Cache	404.38 (60.99×)	79.30
MBPP	LLaDA-1.5	7.71 (1.0×)	38.40
	+ Cache	18.64 (2.42×)	39.40
	+ DC-Leap	410.07 (53.19×)	39.40
	+ DC-Leap + Cache	809.71 (105.02×)	36.40

Impact of draft threshold τ_d . Similarly, we analyze the impact of τ_d on accuracy and throughput in Figures 5 (b, d), while fixing τ_c to 0.70. The performance on both benchmarks follows a consistent pattern: Lowering τ_d compromises accuracy slightly. Conversely, increasing τ_d slightly reduces throughput due to stricter draft selection, as seen in MBPP, where TPS adjusts from 185.19 to 171.35. To ensure throughput and consistent performance across tasks, we identify $\tau_d = 0.98$ as the optimal value.

Impact of maximum window size L . We investigate the sensitivity of the maximum window size L on the GSM8K benchmark, varying from 32 to 128. As illustrated in Figure 6, expanding the window size significantly enhances parallelism. While further increasing to 128 maximizes inference speed, reaching a peak throughput of 54.27 TPS, at the cost of a slight accuracy decline from 80.74% to 79.98%. Given that the performance degradation is negligible relative to the substantial efficiency gains, we adopt $L = 128$ as the default setting to prioritize acceleration.

5. Conclusion

In this work, we identify the strict thresholding driven by the conditional independence assumption in non-contiguous decoding as a critical bottleneck limiting inference speed. While sequential decoding theoretically resolves this issue, it remains inefficient and underutilizes bidirectional context. Building on these insights, we propose DC-Leap, a simple

yet effective method that introduces Dynamic Contiguous Verification and leverages the inherent bidirectional attention of dLLMs via Draft-guided Decoding to unlock the acceleration potential within lower-confidence regimes. As a plug-and-play method, DC-Leap is orthogonal to existing KV Cache optimizations, allowing their acceleration effects to be stacked for synergistic gains.

Impact Statement

This paper presents work whose goal is to advance the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

Acknowledgement

This work was supported by the Guangdong Basic and Applied Basic Research Foundation (2025A1515011546) and by the Shenzhen Science and Technology Program (JCYJ20240813105901003, ZDCY20250901113000001).

References

- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Bai, S., Cai, Y., Chen, R., Chen, K., Chen, X., Cheng, Z., Deng, L., Ding, W., Gao, C., Ge, C., Ge, W., Guo, Z., Huang, Q., Huang, J., and Fei Huang, e. a. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025.
- Bao, W., Chen, Z., Xu, D., and Shang, Y. Learning to parallel: Accelerating diffusion large language models via learnable parallel decoding. In *The Fourteenth International Conference on Learning Representations*, 2026.
- Ben-Hamu, H., Gat, I., Severo, D., Nolte, N., and Karrer, B. Accelerated sampling from masked diffusion models via entropy bounded unmasking. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026.
- Bolya, D., Fu, C.-Y., Dai, X., Zhang, P., Feichtenhofer, C., and Hoffman, J. Token merging: Your vit but faster. In *The Eleventh International Conference on Learning Representations*, 2023.
- Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J. D., Chen, D., and Dao, T. Medusa: Simple llm inference acceleration framework with multiple decoding heads. In *International Conference on Machine Learning*, 2024.
- Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., and Jumper, J. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., and Greg Brockman, e. a. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Chen, X., Huang, S., Guo, C., Wei, C., He, Y., Zhang, J., Li, H. H., and Chen, Y. DPAd: Efficient diffusion language models with suffix dropout. In *The Fourteenth International Conference on Learning Representations*, 2026a.
- Chen, Z., Wang, W., Cao, Y., Liu, Y., Gao, Z., Cui, E., Zhu, J., Ye, S., Tian, H., Liu, Z., Gu, L., Wang, X., Li, Q., Ren, Y., and Zixuan Chen, e. a. Expanding performance boundaries of open-source multimodal models with model, data, and test-time scaling. *arXiv preprint arXiv:2412.05271*, 2025.
- Chen, Z., Fang, G., Ma, X., Yu, R., and Wang, X. dparallel: Learnable parallel decoding for dlms. In *The Fourteenth International Conference on Learning Representations*, 2026b.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Cui, J., Liu, S., Tian, Z., Zhong, Z., and Jia, J. Reslt: Residual learning for long-tailed recognition. *IEEE transactions on pattern analysis and machine intelligence*, 2022.
- Cui, J., Zhong, Z., Tian, Z., Liu, S., Yu, B., and Jia, J. Generalized parametric contrastive learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023.
- Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., Fan, A., et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.
- Fu, Y., Wang, X., Zhang, H., Tian, Y., and Zhao, J. Deep think with confidence. In *The Fourteenth International Conference on Learning Representations*, 2026.
- Ganesan, M., Segal, S., Aggarwal, A., Sinnadurai, N., Lie, S., and Thangarasa, V. MASSV: Multimodal adaptation and self-data distillation for speculative decoding of vision-language models. In *ES-FoMo III: 3rd Workshop on Efficient Systems for Foundation Models*, 2025.

- Guo, C., Pleiss, G., Sun, Y., and Weinberger, K. Q. On calibration of modern neural networks. In *International conference on machine learning*, 2017.
- Han, X., Kumar, S., and Tsvetkov, Y. Ssd-lm: Semi-autoregressive simplex-based diffusion language model for text generation and modular control. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2023.
- Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- Huang, J., Hu, X., Shi, S., Tian, Z., and Jiang, L. Edit360: 2d image edits to 3d assets from any angle. In *ICCV*, 2025.
- Huang, P., Liu, T., Liu, Z., Yan, Y., Wang, S., Xiao, T., Chen, Z., and Sun, M. Empirical analysis of decoding biases in masked diffusion models. *arXiv:2508.13021*, 2026.
- Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., de las Casas, D., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., Lavaud, L. R., Lachaux, M.-A., Stock, P., Scao, T. L., Lavril, T., Wang, T., Lacroix, T., and Sayed, W. E. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.
- Jiang, L., Shi, S., Tian, Z., Lai, X., Liu, S., Fu, C.-W., and Jia, J. Guided point contrastive learning for semi-supervised point cloud semantic segmentation. In *Proceedings of the IEEE/CVF international conference on computer vision*, 2021.
- Kong, F., Zhang, J., Liu, Y., Wu, Z., Tian, Y., Zhou, G., et al. Accelerating diffusion llm inference via local determinism propagation. *arXiv preprint arXiv:2510.07081*, 2025.
- Kuhn, L., Gal, Y., and Farquhar, S. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In *The Eleventh International Conference on Learning Representations*, 2023.
- Lai, X., Tian, Z., Jiang, L., Liu, S., Zhao, H., Wang, L., and Jia, J. Semi-supervised semantic segmentation with directional context-aware consistency. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021.
- Lai, X., Tian, Z., Chen, Y., Li, Y., Yuan, Y., Liu, S., and Jia, J. Lisa: Reasoning segmentation via large language model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024a.
- Lai, X., Tian, Z., Chen, Y., Yang, S., Peng, X., and Jia, J. Step-dpo: Step-wise preference optimization for long-chain reasoning of llms. *arXiv preprint arXiv:2406.18629*, 2024b.
- Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.
- Li, L., Chen, Z., Chen, G., Zhang, Y., Su, Y., Xing, E., and Zhang, K. Confidence matters: Revisiting intrinsic self-correction capabilities of large language models. *arXiv preprint arXiv:2402.12563*, 2024a.
- Li, P., Zhou, Y., Muhtar, D., Yin, L., Yan, S., Shen, L., Liang, Y., Vosoughi, S., and Liu, S. Diffusion language model knows the answer before it decodes. In *The Fourteenth International Conference on Learning Representations*, 2026.
- Li, X. L., Thickstun, J., Gulrajani, I., Liang, P., and Hashimoto, T. Diffusion-LM improves controllable text generation. In *Advances in Neural Information Processing Systems*, 2022.
- Li, Y., Wei, F., Zhang, C., and Zhang, H. Eagle: Speculative sampling requires rethinking feature uncertainty. In *International Conference on Machine Learning*, 2024b.
- Li, Y., Liu, Z., Li, Z., Zhang, X., Xu, Z., Chen, X., Shi, H., Jiang, S., Wang, X., Wang, J., et al. Perception, reason, think, and plan: A survey on large multimodal reasoning models. *arXiv preprint arXiv:2505.04921*, 2025.
- Liang, Y., Ge, C., Tong, Z., Song, Y., Wang, J., and Xie, P. Not all patches are what you need: Expediting vision transformers via token reorganizations. *arXiv preprint arXiv:2202.07800*, 2022.
- Lin, J., Tang, J., Tang, H., Yang, S., Xiao, G., and Han, S. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. *GetMobile: Mobile Computing and Communications*, 2025.
- Liu, H., Li, C., Li, Y., Li, B., Zhang, Y., Shen, S., and Lee, Y. J. Llava-next: Improved reasoning, ocr, and world knowledge, January 2024. URL <https://llava-vl.github.io/blog/2024-01-30-llava-next/>.
- Liu, Z., Yang, Y., Zhang, Y., Chen, J., Zou, C., Wei, Q., Wang, S., and Zhang, L. dllm-cache: Accelerating diffusion large language models with adaptive caching. *arXiv preprint arXiv:2506.06295*, 2025.
- Luo, X., Tian, Z., Zhang, T., Yu, B., Tang, Y. Y., and Jia, J. Pfenet++: Boosting few-shot semantic segmentation with

- the noise-filtered context-aware prior mask. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023.
- Nie, S., Zhu, F., You, Z., Zhang, X., Ou, J., Hu, J., ZHOU, J., Lin, Y., Wen, J.-R., and Li, C. Large language diffusion models. In *ICLR 2025 Workshop on Deep Generative Model in Machine Learning: Theory, Principle and Efficiency*, 2025.
- Ning, Z., Tian, Z., Lu, G., and Pei, W. Boosting few-shot 3d point cloud segmentation via query-guided enhancement. In *Proceedings of the 31st ACM international conference on multimedia*, 2023.
- Peng, B., Tian, Z., Wu, X., Wang, C., Liu, S., Su, J., and Jia, J. Hierarchical dense correlation distillation for few-shot segmentation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2023.
- Peng, B., Tian, Z., Liu, S., Yang, M., and Jia, J. Scalable language model with generalized continual learning. *arXiv preprint arXiv:2404.07470*, 2024a.
- Peng, B., Wu, X., Jiang, L., Chen, Y., Zhao, H., Tian, Z., and Jia, J. Oa-cnns: Omni-adaptive sparse cnns for 3d semantic segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024b.
- Peng, S., Wang, W., Tian, Z., Yang, S., Wu, X., Xu, H., Zhang, C., Isobe, T., Hu, B., and Zhang, M. Omni-dpo: A dual-perspective paradigm for dynamic preference learning of llms. *arXiv preprint arXiv:2506.10054*, 2025a.
- Peng, S., Yang, S., Jiang, L., and Tian, Z. Mitigating object hallucinations via sentence-level early intervention. *arXiv preprint arXiv:2507.12455*, 2025b.
- Rao, Y., Zhao, W., Liu, B., Lu, J., Zhou, J., and Hsieh, C.-J. Dynamicvit: Efficient vision transformers with dynamic token sparsification. 2021.
- Shao, T., Tian, Z., Zhao, H., and Su, J. Explore the potential of clip for training-free open vocabulary semantic segmentation. In *European Conference on Computer Vision*, 2024.
- Tian, Z., Shu, M., Lyu, P., Li, R., Zhou, C., Shen, X., and Jia, J. Learning shape-aware embedding for scene text detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019.
- Tian, Z., Zhao, H., Shu, M., Yang, Z., Li, R., and Jia, J. Prior guided feature enrichment network for few-shot segmentation. *IEEE transactions on pattern analysis and machine intelligence*, 2020.
- Tian, Z., Chen, P., Lai, X., Jiang, L., Liu, S., Zhao, H., Yu, B., Yang, M.-C., and Jia, J. Adaptive perspective distillation for semantic segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022a.
- Tian, Z., Lai, X., Jiang, L., Liu, S., Shu, M., Zhao, H., and Jia, J. Generalized few-shot semantic segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022b.
- Tian, Z., Cui, J., Jiang, L., Qi, X., Lai, X., Chen, Y., Liu, S., and Jia, J. Learning context-aware classifier for semantic segmentation. In *Proceedings of the AAAI conference on artificial intelligence*, 2023.
- Wang, C., Jiang, L., Wu, X., Tian, Z., Peng, B., Zhao, H., and Jia, J. Groupcontrast: Semantic-aware self-supervised representation learning for 3d understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.
- Wang, J., Chen, B., Li, Y., Kang, B., Chen, Y., and Tian, Z. Declip: Decoupled learning for open-vocabulary dense perception. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025a.
- Wang, J., Chen, K., Li, Y., Chen, B., Zhao, H., Qi, X., and Tian, Z. Generalized decoupled learning for enhancing open-vocabulary dense perception. *arXiv preprint arXiv:2508.11256*, 2025b.
- Wu, C., Zhang, H., Xue, S., Liu, Z., Diao, S., Zhu, L., Luo, P., Han, S., and Xie, E. Fast-dLLM: Training-free acceleration of diffusion LLM by enabling KV cache and parallel decoding. In *The Fourteenth International Conference on Learning Representations*, 2026.
- Wu, X., Tian, Z., Wen, X., Peng, B., Liu, X., Yu, K., and Zhao, H. Towards large-scale 3d representation learning with multi-dataset point prompt training. In *CVPR*, 2024.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025a.
- Yang, S., Qu, T., Lai, X., Tian, Z., Peng, B., Liu, S., and Jia, J. Lisa++: An improved baseline for reasoning segmentation with large language model. *arXiv preprint arXiv:2312.17240*, 2023.
- Yang, S., Tian, Z., Jiang, L., and Jia, J. Unified language-driven zero-shot domain adaptation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2024.
- Yang, S., Chen, Y., Tian, Z., Wang, C., Li, J., Yu, B., and Jia, J. Visionzip: Longer is better but not necessary in vision language models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025b.

- Ye, J., Xie, Z., Zheng, L., Gao, J., Wu, Z., Jiang, X., Li, Z., and Kong, L. Dream 7b: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025.
- Zhang, Y., Wu, X., Lao, Y., Wang, C., Tian, Z., Wang, N., and Zhao, H. Concerto: Joint 2d-3d self-supervised learning emerges spatial representations. In *NeurIPS*, 2025.
- Zhou, J., Lu, T., Mishra, S., Brahma, S., Basu, S., Luan, Y., Zhou, D., and Hou, L. Instruction-following evaluation for large language models. *CoRR*, 2023.
- Zhu, F., Wang, R., Nie, S., Zhang, X., Wu, C., Hu, J., Zhou, J., Chen, J., Lin, Y., Wen, J.-R., et al. Llada 1.5: Variance-reduced preference optimization for large language diffusion models. *arXiv preprint arXiv:2505.19223*, 2025.

Contents

A	Algorithm of DC-Leap	14
B	Related Work	14
B.1	Inference Acceleration in LLMs and VLMs	14
B.2	Diffusion Large Language Models	15
B.3	Sampling and Acceleration Strategies of dLLMs	15
C	Case Study	16
D	Analysis of Decoding Dynamics	18
E	Analysis of Continuous Drafts	18
E.1	Motivation and Hypothesis	18
E.2	Experimental Setup	18
E.3	Results	19
E.4	Observations and Discussions	19
F	More Results and Analysis	20
F.1	More Results of Ablations	20
F.2	More Results of Step Distilled dLLMs	21
F.3	Analysis of Alternative Confidence Definition	21

A. Algorithm of DC-Leap

The pseudocode of DC-Leap is shown in Algorithm 1.

Algorithm 1 DC-Leap: Draft-Guided Contiguous Parallel Decoding

Require: Model p_θ , prompt $\mathbf{x}_{\text{prompt}}$, generation length L_g , max window size L , confidence thresholds $\tau_{\text{commit}}, \tau_{\text{draft}}$

- 1: Initialize sequence $\mathbf{x} \leftarrow \text{concat}(\mathbf{x}_{\text{prompt}}, \{[\text{MASK}]\}^{L_g})$
- 2: $p \leftarrow \text{len}(\mathbf{x}_{\text{prompt}})$ ▷ Pointer to the first unverified position
- 3: **while** $p < L_g$ **do**
- 4: **// Draft Invalidation**
- 5: $\mathbf{x}_{p:p+L} \leftarrow \{[\text{MASK}]\}^L$
- 6: **// Forward Pass**
- 7: $\hat{\mathbf{x}}, \mathbf{c} \leftarrow \text{Forward}(p_\theta, \mathbf{x})$
- 8: **// Dynamic Contiguous Verification**
- 9: $K \leftarrow \sum_{i=0}^{L-1} \prod_{j=0}^i \mathbb{I}(c_{p+j} > \tau_{\text{commit}})$ ▷ Caculate the length of dynamic decoding window
- 10: **// Commit Verified Tokens & Update Draft Placeholders**
- 11: $\mathbf{x}_{p:p+K} \leftarrow \hat{\mathbf{x}}_{p:p+K}$ ▷ Commit verified tokens
- 12: **for** $j = p + L$ **to** L_g **do**
- 13: **if** $c_j > \tau_{\text{draft}}$ **then**
- 14: $x_j \leftarrow \hat{x}_j$ ▷ Update future drafts
- 15: **end if**
- 16: **end for**
- 17: $p \leftarrow p + K$ ▷ Dynamically advance
- 18: **end while**
- 19: **return** $\mathbf{x}$

B. Related Work

B.1. Inference Acceleration in LLMs and VLMs

Inference efficiency remains a critical bottleneck for both Large Language Models (LLMs) and Vision Language Models (VLMs).

In autoregressive (AR) LLMs such as Llama 3 (Dubey et al., 2024), Qwen 3 (Yang et al., 2025a) and Mistral 7B (Jiang et al., 2023), acceleration strategies primarily target the sequential bottleneck of the decoding process. While advancements in preference optimization and continual learning bolster complex reasoning capabilities (Lai et al., 2024b; Peng et al., 2024a; 2025a), inference acceleration has also become increasingly vital. Speculative decoding emerges as a representative paradigm, where a lightweight drafter predicts multiple future tokens that are subsequently verified in parallel by a larger target model (Leviathan et al., 2023; Chen et al., 2023). Building on this concept, recent works have introduced multi-head structures and tree-based verification to further expand the parallel search space during generation (Cai et al., 2024; Li et al., 2024b). Additionally, quantization and pruning techniques reduce the memory footprint and computational intensity, allowing models to achieve higher throughput on resource-constrained hardware (Frantar et al., 2022; Lin et al., 2025).

For VLMs, such as Qwen3-VL (Bai et al., 2025), LLaVa-Next (Liu et al., 2024) and InternVL 2.5 (Chen et al., 2025), inference acceleration is further complicated by the massive number of visual tokens processed through cross-modal attention. Current research addresses this by identifying and removing redundant visual information via token pruning or merging strategies (Bolya et al., 2023; Liang et al., 2022; Yang et al., 2025b). Dynamic resolution techniques also allow models to allocate more computational resources to informative image regions while processing background areas at a lower cost (Rao et al., 2021). Furthermore, multimodal speculative decoding has been proposed to exploit the correlation between visual features and linguistic outputs, enabling the parallel drafting of vision-textual sequences (Ganesan et al., 2025). Such efficiencies are vital for advancing large multimodal reasoning models (Li et al., 2025), which have recently expanded to complex tasks like reasoning segmentation (Lai et al., 2024a; Yang et al., 2023) and object hallucination mitigation (Peng et al., 2025b). While these methods effectively mitigate the overhead of AR architectures, they remain bound by the causal requirement of sequential drafting. In contrast, dLLMs provide a framework for fully parallel decoding, which can theoretically achieve superior speedsups.

Moreover, the principles of efficient contextual processing and robust representation learning are critical across broader computer vision domains. This includes optimizing feature extraction for few-shot and semi-supervised semantic segmentation (Tian et al., 2020; Lai et al., 2021; Tian et al., 2022b; Peng et al., 2023; Luo et al., 2023), refining adaptive and context-aware classifiers for long-tailed scenarios (Tian et al., 2022a; 2023; Cui et al., 2022), and advancing training-free or open-vocabulary dense perception alongside cross-domain adaptation and generalized contrastive learning (Shao et al., 2024; Yang et al., 2024; Wang et al., 2025a;b; Cui et al., 2023). Early efforts have also explored efficient shape-aware embeddings for specific downstream tasks like text detection (Tian et al., 2019). Similarly, in 3D understanding, parallel challenges in computational overhead have spurred the development of sparse and adaptive architectures for point cloud segmentation (Jiang et al., 2021; Ning et al., 2023; Peng et al., 2024b), alongside scalable multi-dataset prompt training (Wu et al., 2024) and joint 2D-3D self-supervised representation frameworks (Wang et al., 2024; Zhang et al., 2025). Efficient representations in these spaces are also key to downstream generation tasks, such as multi-angle 3D asset editing (Huang et al., 2025).

B.2. Diffusion Large Language Models

The paradigm of generative diffusion models, originally dominant in continuous domains such as image synthesis, has recently been adapted to the discrete nature of natural language. Early attempts, such as Diffusion-LM (Li et al., 2022), mapped discrete tokens to a continuous embedding space to apply Gaussian diffusion, though this often resulted in suboptimal text generation quality due to rounding errors. SSD-LM (Han et al., 2023) improved upon this by performing diffusion on the probability simplex. However, the field has largely converged towards Masked Diffusion Models (MDMs), which operate directly in the discrete state space by treating the forward process as a random masking operation.

Recent scaling efforts have demonstrated that dLLMs can rival the performance of autoregressive (AR) baselines. LLaDA (Nie et al., 2025; Zhu et al., 2025) and Dream (Ye et al., 2025) represent the state-of-the-art in this domain, scaling masked diffusion to billions of parameters. Unlike AR models which are constrained by causal attention, these models utilize bidirectional attention, allowing every token to attend to the entire sequence context during inference. While this architecture theoretically supports parallel decoding, the standard sampling process, requiring tens or hundreds of iterative refinement steps, remains a significant bottleneck for latency-sensitive applications.

B.3. Sampling and Acceleration Strategies of dLLMs

For dLLMs, acceleration strategies must navigate a different trade-off between parallelism and consistency. Standard samplers like Fast-dLLM (Wu et al., 2026) introduce training-free acceleration by implementing a confidence-aware parallel decoding strategy, where tokens exceeding a certain probability threshold are accepted simultaneously. Building on this, LocalLeap (Kong et al., 2025) observes a locality property in decoding and employs high-confidence tokens as anchors to dynamically relax thresholds for neighboring tokens. DPAd (Chen et al., 2026a) further optimizes efficiency by applying suffix dropout and sliding windows to prune redundant computations in the suffix region.

Concurrently, several recent works have explored more sophisticated adaptive policies for accelerating dLLM inference. UNCODE (Huang et al., 2026) addresses decoding biases by calibrating confidence scores with positional priors and informativeness priors to mitigate rigid boundary bias and trivial token bias. Prophet (Li et al., 2026) introduces an early-commit mechanism that uses the confidence gap between top-two candidates as a criterion to dynamically terminate denosing steps once predictions stabilize. Moving beyond fixed heuristics, Learn2PD (Bao et al., 2026) proposes a post-training framework that employs a lightweight MLP filter to adaptively predict token stability and optimize remasking decisions.

However, a critical limitation in these parallel approaches is the independence assumption. As noted by the EB-Sampler (Ben-Hamu et al., 2026), independently unmasking multiple tokens introduces a joint probability dependence error, as the validity of one token often depends on the specific instantiation of its neighbors. To mitigate this, prior methods like Fast-dLLM and LocalLeap are forced to adopt conservative, high-confidence thresholds (e.g., > 0.9) to ensure safety, thereby sacrificing the potential speedup from correctly predicted but lower-confidence tokens. Our work addresses this limitation by enforcing a causal, left-to-right verification order, which eliminates the joint dependence error and unlocks the stable utilization of lower confidence predictions.

C. Case Study

We evaluate the practical impact of DC-Leap through a case study on LLaDA-8B-Instruct, LLaDA-1.5, and Dream-v0-7B-Instruct. Table 6 presents the results both with and without DC-Leap integration. All experiments are conducted on NVIDIA RTX PRO 6000 Blackwell, with a generation length of 256, a block length of 32, and a maximum decoding window size set to half of the total generation length. Using a commitment threshold $\tau_c = 0.65$ and a draft threshold $\tau_d = 0.95$, empirical results show that DC-Leap achieves substantial wall-clock speedups while maintaining reasoning accuracy across all models.

Interestingly, while the final answers remain identical to the baselines, DC-Leap often yields more concise reasoning paths. This suggests that our draft-guided mechanism allows the model to incorporate future high-confidence tokens through bidirectional attention, providing a global context that guides the model to bypass redundant steps and steer the denoising trajectory towards a more efficient logical sequence. Furthermore, the absence of hallucinations even with the lower commitment threshold ($\tau_c = 0.65$) demonstrates that our contiguous constraint effectively stabilizes the generation process, ensuring that the leaps in decoding remain logically grounded and instruction-compliant.

Table 6. Case Studies across LLaDA-8B-Instruct, LLaDA-1.5, and Dream-v0-7B-Instruct.

Setting	Content
Case 1: LLaDA-8B-Instruct	
Prompt	Ellie has found an old bicycle in a field and thinks it just needs some oil to work well again. She needs 10ml of oil to fix each wheel and will need another 5ml of oil to fix the rest of the bike. How much oil does she need in total to fix the bike?
LLaDA-8B-Instruct	Latency: 9.84s <i>Output:</i> To determine the total amount of oil Ellie needs to fix the bike, we can break down the problem into two parts: the oil needed for the wheels and the oil needed for the rest of the bike. Step 1: Calculate the oil needed for the wheels. Ellie needs 10ml of oil to fix each wheel. Since a bicycle has two wheels, the total oil needed for the wheels is: $10 \text{ ml/wheel} \times 2 \text{ wheels} = 20 \text{ ml}$ Step 2: Add the oil needed for the rest of the bike. Ellie needs an additional 5ml of oil to fix the rest of the bike. Therefore, the total amount of oil needed is: $20 \text{ ml} + 5 \text{ ml} = 25 \text{ ml}$ So, the total amount of oil Ellie needs to fix the bike is 25 ml.
+DC-Leap	Latency: 1.95s <i>Output:</i> To determine the total amount of oil Ellie needs to fix the bike, we can break down the problem into smaller steps: Step 1: Calculate the amount of oil needed for the wheels. Ellie needs 10ml of oil to fix each wheel. Since a bicycle has two wheels, we multiply the amount of oil needed for one wheel by 2: $10 \text{ ml/wheel} \times 2 \text{ wheels} = 20 \text{ ml}$ Step 2: Add the amount of oil needed for the rest of the bike. Ellie needs an additional 5ml of oil to fix the rest of the bike. We add this to the amount of oil needed for the wheels: $20 \text{ ml} + 5 \text{ ml} = 25 \text{ ml}$ Therefore, the total amount of oil Ellie needs to fix the bike is 25 ml.
Case 2: LLaDA-1.5	
Prompt	Ellie has found an old bicycle in a field and thinks it just needs some oil to work well again. She needs 10ml of oil to fix each wheel and will need another 5ml of oil to fix the rest of the bike. How much oil does she need in total to fix the bike?

Continued on next page...

Table 6 – continued from previous page

Setting	Content
LLaDA-1.5	Latency: 9.60s To determine the total amount of oil Ellie needs to fix the bike, we can break down the problem into two parts: the oil needed for the wheels and the oil needed for the rest of the bike. Step 1: Calculate the oil needed for the wheels Ellie needs 10ml of oil for each wheel. Since a bicycle has two wheels, the total oil needed for the wheels is: $10 \text{ ml/wheel} \times 2 \text{ wheels} = 20 \text{ ml}$ Step 2: Calculate the oil needed for the rest of the bike. Ellie needs 5ml of oil for the rest of the bike. Therefore, the total oil needed for the rest of the bike is: 5 ml Step 3: Add the oil needed for the wheels and the rest of the bike to find the total amount of oil needed. $20 \text{ ml} + 5 \text{ ml} = 25 \text{ ml}$ So, the total amount of oil Ellie needs in total is 25 ml.
+DC-Leap	Latency: 1.71s To determine the total amount of oil Ellie needs to fix the bike, we can break down the problem into smaller steps: Step 1: Calculate the amount of oil needed for the wheels. Ellie needs 10ml of oil to fix each wheel. Assuming the bicycle has two wheels, the total amount of oil needed for the wheels is: $10 \text{ ml/wheel} \times 2 \text{ wheels} = 20 \text{ ml}$ Step 2: Add the amount of oil needed for the rest of the bike. Ellie needs an additional 5ml of oil to fix the rest of the bike. Therefore, the total amount of oil needed is: $20 \text{ ml} + 5 \text{ ml} = 25 \text{ ml}$ So, the total amount of oil Ellie needs to fix the bike is 25 ml.
Case 3: Dream-v0-7B-Instruct	
Prompt	Ellie has found an old bicycle in a field and thinks it just needs some oil to work well again. She needs 10ml of oil to fix each wheel and will need another 5ml of oil to fix the rest of the bike. How much oil does she need in total to fix the bike?
Dream-v0-7B-Instruct	Latency: 8.30s Ellie needs 10ml of oil to fix each wheel, and there are 2 wheels on a bicycle. So, she needs $10\text{ml} \times 2 = 20\text{ml}$ of oil for the wheels. Additionally, she needs 5ml of oil to fix the rest of the bike. Therefore, the total amount of oil Ellie needs is $20\text{ml} + 5\text{ml} = 25\text{ml}$. Ellie needs 25 ml of oil in total to fix the bike.
+DC-Leap	Latency: 1.32s Ellie needs 10ml of oil to fix each wheel, so she needs $10\text{ml} \times 2 = 20\text{ml}$ of oil for the wheels. She also needs 5ml of oil to fix the rest of the bike. Therefore, Ellie needs a total of $20\text{ml} + 5\text{ml} = 25\text{ml}$ of oil to fix the bike. ##### 25 The answer is: 25

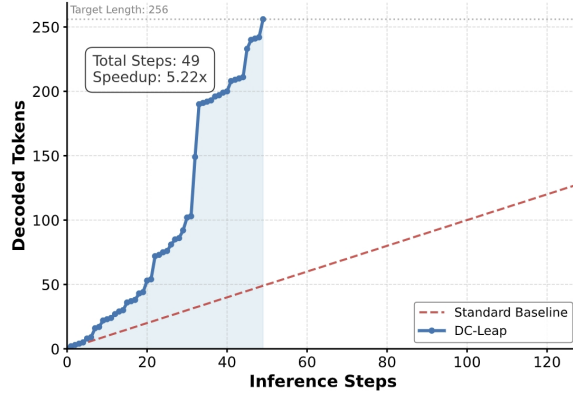


Figure 7. **Cumulative decoded tokens with inference steps advancing.** DC-Leap (blue) shows a bursty decoding pattern, enabling massive jumps in generation progress (e.g., around step 35) compared to the linear standard baseline (red). This sample achieves a total speedup of $5.22\times$ on a 256-token task.

D. Analysis of Decoding Dynamics

To further understand the acceleration mechanism of DC-Leap, we analyze the cumulative number of decoded tokens across inference steps. We perform this analysis on a random GSM8K sample with a generation length of 256 tokens.

Figure 7 visualizes the decoding trajectory of DC-Leap compared to the standard diffusion baseline. Compared to baseline, DC-Leap exhibits a non-linear, bursty generation pattern. We observe three distinct phases in the DC-Leap trajectory:

Exploration Phase (Steps 0-20). In the initial steps, the slope is relatively shallow as DCV strictly improves the acceptance quality.

Acceleration Phase (Steps 20-35). A nearly vertical surge in decoded tokens occurs around step 22, showcasing the synergy between drafts and the Dynamic Contiguous Verification (DCV) mechanism. In this phase, drafts serve as future anchors, progressively stabilizing the context in the decoding window. DCV capitalizes on this draft-guided coherence by applying a relaxed confidence threshold over continuous token spans. This integration allows the model to prioritize structural continuity, enabling effectively leaping through the generation process.

Convergence Phase (Steps 35-49). The generation completes rapidly, achieving the target length of 256 tokens in just 49 steps.

This analysis reveals DC-Leap as a dynamic, intelligent acceleration strategy centered on Dynamic Contiguous Verification and Draft mechanism. By enforcing a structural continuity constraint, this mechanism ensures generation quality even at lower confidence thresholds, enabling the simultaneous commitment of extended token spans. During the Acceleration Phase, this strategy effectively capitalizes on the rising confidence across the decoding window to leap through the sequence with quality drafts, facilitating convergence, significantly accelerating through massive, verified parallel updates.

E. Analysis of Continuous Drafts

E.1. Motivation and Hypothesis

In the main methodology of DC-Leap, we demonstrate that enforcing a contiguous verification constraint on the prefix allows us to safely lower the commitment threshold without compromising generation quality. Building on this, a natural hypothesis arises: *Can we extend this contiguous constraint to the Draft mechanism to enable lower draft thresholds?*

Intuitively, if we enforce that drafts must also form a continuous chain starting from the verification boundary, we might be able to safely relax the draft threshold τ_{draft} significantly. Therefore, this could allow the model to leap further into the future with a denser draft sequence, potentially yielding higher speedups without compromising performance.

E.2. Experimental Setup

To test this hypothesis, we implement a variant of DC-Leap with Contiguous Drafts. Unlike the standard Draft mechanism, where any future token exceeding τ_{draft} is cached regardless of intermediate gaps, the Contiguous Draft strategy updates the bank if and only if:

$$d_j \leftarrow \hat{x}_j \quad \text{iff} \quad \prod_{k=L_m+L_{\text{prefix}}}^j \mathbb{I}(c_{t+k} > \tau_{\text{draft}}) = 1, \quad (9)$$

where L_m and L_{prefix} denote the maximum window size and the prefix length in DCV, respectively. And $\hat{x}_j$ represents the candidate token generated at position j , while d_j denotes the finalized draft token accepted for the current decoding iteration. We evaluate this variant on GSM8K using LLaDA-1.5 with a fixed $\tau_{\text{commit}} = 0.7$ and varying τ_{draft} from 1.00 down to 0.00.

E.3. Results

The experimental results are presented in Table 7.

Table 7. **Performance of Contiguous Draft Strategy on GSM8K.** We fix the commitment threshold $\tau_{\text{commit}} = 0.7$ and vary the draft threshold τ_{draft} .

τ_{draft}	Throughput (TPS)	Speedup	Accuracy (%)
1.00	51.73	4.89×	79.53
0.95	52.10	4.92×	79.53
0.80	51.72	4.89×	79.53
0.70	51.89	4.90×	79.53
0.65	50.96	4.82×	79.53
0.60	51.97	4.91×	79.53
0.50	51.92	4.91×	79.61
0.40	51.17	4.84×	78.85
0.30	52.43	4.96×	78.39
0.20	51.71	4.89×	76.19
0.10	54.31	4.95×	63.15
0.00	62.49	5.91×	59.82

E.4. Observations and Discussions

Observations. The results reveal three distinct behavioral phases, offering experimental evidence against the contiguous drafting mechanism:

- **Stagnant Phase** ($\tau_{\text{draft}} \in [0.60, 1.00]$): Accuracy remains strictly invariant at 79.53%, and throughput stagnates around 51 ~ 52 TPS. This plateau suggests that the contiguous constraint severely restricts the length of generated drafts, making it virtually impossible to form a sequence that extends beyond the dynamic decoding window. Consequently, no effective drafts are cached, rendering the mechanism functionally equivalent to the baseline.
- **Degradation Phase** ($\tau_{\text{draft}} \in [0.20, 0.60]$): As the threshold is lowered further, accuracy begins to decline without yielding throughput gains. This suggests that while longer draft chains are finally being formed, they inevitably contain low-quality tokens due to the lower confidence threshold. These poor-quality drafts act as noise rather than informative anchors, confusing the model without facilitating convergence.
- **Collapse Phase** ($\tau_{\text{draft}} \in [0, 0.20]$): Only at extremely low thresholds does throughput perceptibly increase, but this comes at the cost of a catastrophic accuracy collapse. In this range, the model blindly accepts incorrect drafts, leading to rapid but nonsensical generation.

Discussions. These findings expose a fundamental conflict between the draft reliability of dLLMs and the contiguous constraint:

- **The Domino Effect:** The cumulative product operation enforces a domino effect: a single low-confidence token truncates the entire future draft chain. To overcome this truncation and generate a draft long enough to extend beyond the dynamic decoding window, it becomes necessary to drastically reduce τ_{draft} . However, doing so inevitably introduces erroneous tokens into the future context. Thus, contiguous drafting faces an unsolvable dilemma: the high thresholds required for guaranteeing quality and the low thresholds needed for ensuring drafts’ length are mutually exclusive.
- **Failure to Provide Informative References:** The fundamental purpose of the draft-guided decoding strategy is to provide reliable semantic anchors that the bidirectional attention mechanism can attend to, thereby resolving uncertainty in the current masked region. However, high-confidence tokens often appear scattered in the future context. By enforcing continuity, we discard this valuable information. This confirms that the draft strategy we proposed in Section 3 is superior: it prioritizes the quality and informativeness of the future context over its continuity, ensuring that the drafts serve as valid references rather than noise.

Table 8. Ablations of commitment threshold τ_c across three models.

Metric	Model	0.65	0.68	0.70	0.72	0.75	0.80
Accuracy	LLaDA-8B-Instruct	76.19	76.95	76.65	77.03	77.10	77.48
	LLaDA-1.5	79.08	79.98	80.21	80.59	81.20	81.20
	Dream-v0-7B-Instruct	73.09	73.77	73.92	74.15	74.37	74.45
TPS	LLaDA-8B-Instruct	54.64	52.39	50.22	48.12	45.51	41.46
	LLaDA-1.5	57.84	54.86	52.07	50.65	48.13	44.24
	Dream-v0-7B-Instruct	80.87	77.21	75.87	73.60	69.86	65.54

 Table 9. Ablations of draft threshold τ_d across three models.

Metric	Model	0.85	0.88	0.90	0.92	0.95	0.98
Accuracy	LLaDA-8B-Instruct	77.48	77.03	77.48	77.86	77.26	76.65
	LLaDA-1.5	78.77	79.76	79.68	80.36	79.98	80.21
	Dream-v0-7B-Instruct	71.87	72.02	72.40	73.09	73.39	73.92
TPS	LLaDA-8B-Instruct	48.66	49.15	49.90	49.77	50.00	52.39
	LLaDA-1.5	57.45	55.96	54.96	54.89	54.27	52.07
	Dream-v0-7B-Instruct	74.90	75.18	75.08	75.48	74.55	75.87

F. More Results and Analysis

F.1. More Results of Ablations

In this section, we extend our sensitivity analysis to all three models: LLaDA-8B-Instruct (Nie et al., 2025), LLaDA-1.5 (Zhu et al., 2025), and Dream-v0-7B-Instruct (Ye et al., 2025). The results (detailed in Table 8, Table 9, and Table 10) on the GSM8K benchmark with a fixed generation length of 256 illustrate the trade-offs between generation quality and efficiency, providing a clearer understanding of our design choices.

Sensitivity to commitment threshold τ_c . Firstly, we fix $\tau_d = 0.98$ and vary τ_c from 0.65 to 0.80. As shown in Table 8, across all models, increasing τ_c consistently improves accuracy but leads to a reduction in TPS. Overall, accuracy remains consistently high and stable across all models, demonstrating that DC-Leap is robust to the choice of τ_c within the above range.

Sensitivity to draft threshold τ_d . For draft threshold τ_d , we fix the commitment threshold τ_c at 0.70 and vary τ_d from 0.65 to 0.8 τ_d across three models. From Table 9, we can observe that LLaDA-8B-Instruct exhibits an inverse trend in both accuracy and TPS compared to LLaDA-1.5 and Dream-v0-7B-Instruct, which can be analyzed through their distinct training paradigms.

LLaDA-8B-Instruct is trained from scratch with a mask-diffusion objective; it is inherently robust to non-contiguous and fragmented contexts. For this model, a lower τ_d aligns better with its pre-training distribution, which can facilitate faster and more reliable generation, even if individual draft tokens are less precise.

While sharing the same backbone as LLaDA-8B-Instruct, LLaDA-1.5 is refined via VRPO to pursue high-reward and deterministic outputs, making it more sensitive to the quality of right-side contexts. So a higher τ_d guarantees high-quality drafts, leading to an improvement in both accuracy and TPS.

Dream-v0-7B-Instruct inherits its weights from an autoregressive model with a strong left-to-right causal prior. This architectural heritage makes it more sensitive to low-quality non-causal drafts. A higher τ_d ensures a more precise right-side context, resulting in better performance.

To sum up, for LLaDA-8B-Instruct, the accuracy remains relatively stable across different τ_d with acceptable losses, whereas LLaDA-1.5 and Dream-v0-7B-Instruct are more sensitive to this parameter. Therefore, $\tau_d = 0.98$ serves as an ideal balance that works well for all these models. In addition, all three models exhibit relatively stable performance in both accuracy and TPS.

Sensitivity to maximum window length L : For the maximum window size L , we fix τ_c at 0.70 and τ_d at 0.98, and Table 10 records the accuracy and TPS in different τ_c in three models. In general, all of three models show high stability regarding the window size L . As L increases from 32 to 128, the accuracy remains steady, while the TPS shows a relatively clear gain. This confirms that our method’s efficiency scales well with the window size across different architectures.

Table 10. Ablations of maximum window size L across three models.

Metric	Model	32	64	96	128
Accuracy	LLaDA-8B-Instruct	76.12	76.57	76.65	76.65
	LLaDA-1.5	80.59	80.74	80.29	80.21
	Dream-v0-7B-Instruct	74.15	74.07	74.15	73.92
TPS	LLaDA-8B-Instruct	49.26	49.91	49.70	52.39
	LLaDA-1.5	29.60	51.92	51.08	52.07
	Dream-v0-7B-Instruct	70.34	74.22	75.16	75.87

Table 11. Performance on dParallel-LLaDA-8B-instruct (Chen et al., 2026b).

Model	Metric	GSM8K-CoT (0 shot)	HumanEval (0 shot)	MATH (4 shot)	MBPP (3 shot)
dParallel-LLaDA-8B-instruct	Accuracy	75.06	39.00	35.84	39.40
	TPS	186.11 (1.0 $\times$)	76.38 (1.0 $\times$)	65.23 (1.0 $\times$)	120.53 (1.0 $\times$)
+DC-Leap	Accuracy	74.75	37.42	34.48	41.20
	TPS	256.54 (1.38 $\times$)	141.60 (1.85 $\times$)	95.70 (1.47 $\times$)	290.10 (2.40 $\times$)

Actually, the observed stability in accuracy and the increase in TPS as L expands are consistent with the design intuition of our Dynamic Contiguous Verification (DCV) mechanism. By enforcing a strictly sequential and contiguous verification, DCV ensures that only parallel tokens satisfying the sequential chain are accepted. This decouples generation quality from the maximum window size L to some extent, since accuracy is the same once L is large enough to cover all tokens that pass the DCV. This also explains why the actual speedup exhibits diminishing marginal improvements as L increases.

F.2. More Results of Step Distilled dLLMs

dParallel (Chen et al., 2026b) is a distilled dLLM specifically optimized to enhance the certainty of parallel token prediction, thereby unlocking the acceleration potential of generation. By leveraging a significantly lower confidence threshold than conventional parallel decoding, typically 0.5 in most benchmarks and 0.45 in others, dParallel achieves substantial inference speedups without compromising generative quality.

All experiments are conducted on NVIDIA RTX 6000 Blackwell GPUs. We strictly adhere to the original configurations of dParallel for reproduction. For the integration of DC-Leap, we utilize a consistent hyperparameter set across both dParallel-LLaDA-8B-Instruct and dParallel-Dream-7B-Instruct: $\tau_c = 0.75$, $\tau_d = 0.98$, and maximum window size $L = 128$. The performance, throughput (TPS), and relative speedups are summarized in the Table 11 and Table 12.

It is worth noting that since dParallel already minimizes sampling steps to a highly compressed range for most inputs, further step reduction typically leads to performance degradation. However, DC-Leap demonstrates its robustness in this constrained scenario, successfully delivering additional throughput gains with comparable accuracy.

F.3. Analysis of Alternative Confidence Definition

In this section, we consider alternative confidence definitions beyond Top-1 probability, including calibrated confidence (Fu et al., 2026), entropy-based uncertainty (Kuhn et al., 2023) and self-assessed confidence (Li et al., 2024a). Specifically, we conduct an analysis integrating the three confidence definitions mentioned above into DC-Leap across all three models. The results are summarized in Table 13, Table 14, Table 15 and Table 16. Our results indicate that DC-Leap benefits from the Top-1 confidence formulation, as it demonstrates the strongest synergy with our DCV mechanism.

Analysis of Calibrated Confidence and Semantic Entropy. We evaluate these as alternative confidence estimators by substituting our Top-1 strategy across three models and four benchmarks. As shown in Table 13, Table 14 and Table 15, although both alternatives yield marginal accuracy improvements over Top-1, they significantly reduce the achieved speedup.

We attribute this to the strictness of our DCV mechanism: these more sophisticated estimators impose tighter threshold criteria, reducing the number of tokens committed per forward pass. This suggests that an ideal confidence definition for DC-Leap should not only enhance individual token acceptance accuracy but also remain flexible enough to permit acceptance of more tokens in sequence. Currently, the Top-1 confidence formulation balances these two objectives.

Table 12. Performance on dParallel-Dream-7B-Instruct (Chen et al., 2026b).

Model	Metric	GSM8K-CoT (0 shot)	HumanEval-Instruct (0 shot)	MATH (0 shot)	MBPP-Instruct (0 shot)
dParallel-Dream-7B-Instruct	Accuracy	82.94	54.88	39.88	55.20
	TPS	190.16 (1.0 $\times$)	127.03 (1.0 $\times$)	70.61 (1.0 $\times$)	241.51 (1.0 $\times$)
+DC-Leap	Accuracy	80.52	53.05	37.48	54.00
	TPS	257.28 (1.35 $\times$)	206.27 (1.62 $\times$)	90.32 (1.28 $\times$)	256.54 (1.06 $\times$)

Table 13. Performance comparison on LLaDA-8B-Instruct using three confidence definitions. We compare the Top-1 Confidence baseline with Calibrated Confidence and Semantic Entropy. Accuracy (%) and TPS are reported.

Method	Metric	GSM8K	MBPP	HumanEval	IFEval
Top-1	Accuracy	76.65	29.60	35.37	71.34
	TPS	50.22 (4.80 $\times$)	59.30 (4.58 $\times$)	90.72 (4.73 $\times$)	72.48 (3.44 $\times$)
Calibrated Confidence	Accuracy	77.48	29.80	35.37	70.14
	TPS	25.04 (2.39 $\times$)	30.42 (2.35 $\times$)	40.19 (2.10 $\times$)	45.61 (2.16 $\times$)
Semantic Entropy	Accuracy	77.41	30.0	36.59	71.34
	TPS	28.97 (2.77 $\times$)	32.28 (2.49 $\times$)	32.84 (1.71 $\times$)	24.76 (1.18 $\times$)

Self-Assessed Confidence. Regarding self-assessed confidence, since the IoE-prompt design is specifically tailored for tasks with deterministic answers rather than open-ended generation, we conduct evaluations on the GSM8K benchmark using LLaDA-8B-Instruct and LLaDA-1.5. The results are detailed in Table 16.

Initially, we maintain the original hyperparameters of $\tau_c = 0.7$ and $\tau_d = 0.98$ to ensure a fair comparison with the DC-Leap baseline. We then integrate the IoE-prompt to enable model self-assessment. The results include accuracy, TPS (with speedup ratio), and total runtime seconds, which are summarized in the table below.

As observed, self-assessed confidence is effective, leading to an improvement in both accuracy and TPS. This gain is likely because the self-assessment process enhances the determinism of the preceding context. Consequently, during the subsequent self-evaluation and answer verification phases, a single forward pass can process and verify a larger number of tokens. However, while the overall TPS improves a lot, the total inference time also increases due to the additional reflection process.

Furthermore, since the IoE-prompt inherently provides higher results accuracy, we explore the feasibility of relaxing the confidence threshold. Experimental evidence suggests that τ_c can be safely reduced to 0.6 without compromising accuracy. Under this configuration, the models achieve even higher TPS while remaining competitive in accuracy compared to the Top-1 baseline.

Table 14. Performance comparison on LLaDA-1.5 using three confidence definitions. We compare the Top-1 Confidence baseline with Calibrated Confidence and Semantic Entropy. Accuracy (%), TPS are reported.

Method	Metric	GSM8K	MBPP	HumanEval	IFEval
Top-1	Accuracy	80.21	42.60	38.41	71.82
	TPS	52.07 (4.92 $\times$)	171.35 (13.04 $\times$)	102.74 (5.30 $\times$)	69.67 (3.22 $\times$)
Calibrated Confidence	Accuracy	81.58	40.40	40.24	70.62
	TPS	26.48 (2.50 $\times$)	49.22 (3.76 $\times$)	40.63 (2.10 $\times$)	40.84 (1.89 $\times$)
Semantic Entropy	Accuracy	80.06	43.00	39.02	72.30
	TPS	26.65 (2.52 $\times$)	25.99 (1.98 $\times$)	34.31 (1.77 $\times$)	24.80 (1.15 $\times$)

Table 15. Performance comparison on Dream-v0-7B-Instruct using three confidence definitions. We compare the Top-1 Confidence baseline with Calibrated Confidence and Semantic Entropy. Accuracy (%), TPS are reported.

Method	Metric	GSM8K	MBPP	HumanEval	IFEval
Top-1	Accuracy	73.92	57.60	59.76	51.92
	TPS	75.87 (5.45 $\times$)	88.59 (5.10 $\times$)	116.21 (5.76 $\times$)	110.64 (4.06 $\times$)
Calibrated Confidence	Accuracy	74.45	57.80	59.76	50.00
	TPS	50.23 (3.61 $\times$)	59.46 (3.43 $\times$)	50.12 (2.48 $\times$)	42.32 (1.55 $\times$)
Semantic Entropy	Accuracy	74.30	58.40	60.36	51.92
	TPS	46.72 (3.36 $\times$)	23.30 (1.34 $\times$)	36.08 (1.79 $\times$)	38.90 (1.43 $\times$)

Table 16. Evaluation of Self-Assessed Confidence on GSM8K. We compare the Top-1 Confidence baseline with Self-assessment confidence at different confidence thresholds (τ_c). Accuracy (%), TPS, and total Runtime (s) are reported.

Method	Metric	LLaDA-8B-Instruct	LLaDA-1.5
Top-1	Accuracy	76.65	80.21
	TPS	50.22 (4.80 $\times$)	52.07 (4.92 $\times$)
	Runtime/s	6724.21	6484.92
Self-Assessed ($\tau_c = 0.7$)	Accuracy	79.53	81.05
	TPS	73.50 (7.02 $\times$)	74.99 (7.09 $\times$)
	Runtime/s	9959.15	9844.73
Self-Assessed ($\tau_c = 0.6$)	Accuracy	77.79	79.98
	TPS	89.32 (8.53 $\times$)	90.87 (8.59 $\times$)
	Runtime/s	8167.90	8081.51